Reto 1 · Vivienda joven en Euskadi

Informe técnico de la versión en desarrollo. Orientación basada en rentas municipales observadas; no concede ni deniega ayudas. La entrega integral sigue pendiente de los requisitos no acreditados.

Usuario: servicios de orientación y jóvenes que comparan una referencia territorial con su presupuesto. Objetivo: automatizar referencias trazables, evitar confundir promociones con ofertas y explicar las reglas. El beneficio esperado es facilitar comparaciones; no se ha medido ahorro de tiempo ni impacto social.

Arquitectura: capturas verificadas → preparación → HDFS raw/processed → Kafka → MongoDB → vistas. EMAL → división temporal → pipeline sklearn → modelo conservado → Streamlit → reglas → n8n → ficha local. Los modelos y estadísticas municipales tienen unidades diferentes de un anuncio individual.

Datos y metodología


```
{
  "fuente": {
    "archivo": "emal_municipios.xlsx",
    "url_solicitada": "https://opendata.euskadi.eus/contenidos/estadistica/122417_emal_tabla
s_estad/opendata/EMAL.-Barrios-Municipios.-2016-2026_es.xlsx",
    "fecha_utc": "2026-11-10T14:37:18.098528+00:00",
    "http_status": 200,
    "url_final": "https://opendata.euskadi.eus/contenidos/estadistica/122417_emal_tablas_est
ad/opendata/EMAL.-Barrios-Municipios.-2016-2026_es.xlsx",
    "estado": "descargado",
    "bytes": 256980,
    "sha256": "f4d27d4e8e9bd3688d4e7a454026a4a0884073dc73e3fc1c0da4891a1ad1af5c"
  },
  "unidad": "municipio/trimestre; nuevos contratos EMAL",
  "observaciones": 768,
  "municipios": 19,
  "objetivos_no_publicados_excluidos": 11,
  "exclusion_territorial": "48036 Galdakao: cambio de perimetro",
  "advertencia": "Serie revisada en la fecha de captura; evaluacion retrospectiva, no prueba
de disponibilidad historica en tiempo real. No representa anuncios individuales ni todos los
municipios de Euskadi.",
  "smote": "No aplicable al objetivo continuo de renta; comparacion binaria auxiliar
separada sobre subidas observadas.",
  "particiones": {
    "train": {
      "filas": 601,
      "desde": 8064,
      "hasta": 8095
    },
    "val": {
      "filas": 75,
      "desde": 8096,
      "hasta": 8099
    },
    "test": {
      "filas": 92,
      "desde": 8100,
      "hasta": 8104
    }
  }
}
```


Aprendizaje supervisado de regresión, IA débil especializada en una variable y un territorio. No razona sobre derechos administrativos ni representa expedientes individuales. Pipeline común: imputación, StandardScaler y OneHotEncoder dentro de cada fold temporal. Las entradas son año, trimestre, municipio/provincia y retardos observados. Galdakao se excluye por cambio territorial; los objetivos no publicados no se imputan. La evaluación es retrospectiva sobre una serie revisada.


```
campo,tipo,nulos_pct_train
codigo_municipio,object,0.0
municipio,object,0.0
anio,int64,0.0
trimestre,int64,0.0
renta_media_eur,float64,0.0
numero_contratos,float64,0.0
periodo,int64,0.0
renta_lag1,float64,4.159733777038269
renta_lag4,float64,13.643926788685523
contratos_lag1,float64,2.995008319467554
provincia,object,0.0
```

Comparación extraída de MLflow

Modelo | MAE train | MAE val | RMSE val | R² val | segundos
MLP_2_capas | 15.88 | 24.20 | 31.86 | 0.9370 | 0.080
Random_Forest | 5.92 | 25.15 | 34.76 | 0.9250 | 1.846
MLP_regularizada | 14.78 | 25.42 | 34.03 | 0.9282 | 0.125
MLP_base | 14.22 | 25.78 | 34.38 | 0.9267 | 0.126
XGBoost | 11.64 | 26.52 | 37.05 | 0.9148 | 0.249
Regresion_lineal | 12.76 | 31.17 | 40.58 | 0.8978 | 0.226
MLP_tanh | 16.97 | 35.16 | 50.79 | 0.8400 | 0.139
SVM | 10.60 | 37.86 | 55.56 | 0.8085 | 0.186
KNN | 0.00 | 51.52 | 60.89 | 0.7700 | 0.092

Ganador conservado: MLP_2_capas. MAE de validación: 24.20 EUR; persistencia: 24.63 EUR. La mejora es pequeña; no demuestra superioridad generalizable. No se reselecciona después de observar test. MAE mide error absoluto medio en euros; RMSE penaliza más los errores grandes. R² compara con la media, no es porcentaje de aciertos.

El MLP usa sklearn (base y tres experimentos), no Keras. El enunciado pide MLP sin fijar biblioteca; los ejercicios adjuntos usan Sequential/Dense/Adam y se documentan como diferencia docente. SMOTE no corresponde a regresión; el experimento auxiliar clasifica subidas trimestrales observadas. SHAP utiliza validación y fondo de train, con interpretación no causal.

Evaluación final conservada


```
{
  "modelo": "MLP_2_capas",
  "n_test": 92,
  "mae": 28.66563646053045,
  "rmse": 37.9285330648495,
  "r2": 0.918932902983818,
  "sha256_modelo": "d6f05d11fa6002de53ecaff671b697bd901f63f946b87cc7484701aea76c210a",
  "sha256_test": "f1520fc0580fee119b031252538961e5d62eee68db6a0783c3a5384bc5a65011",
  "evaluaciones": 1,
  "aviso": "Holdout retrospectivo de agregados EMAL; no valida elegibilidad ni decisiones
personales."
}
```


Esta generación solo lee el informe y la figura existentes. No importa joblib, no ejecuta inferencia sobre test ni ajusta hiperparámetros. Las reaperturas de la app reutilizan el informe sellado.

Resultados municipales y límites

```
# Estado de entrega y comprobación por asignatura

## Actualización del 11 de noviembre de 2026

Se conserva la decisión del modelo y el test único. La continuación aporta
129 pruebas unitarias, siete pruebas de MongoDB real y cinco pruebas de
HDFS/Kafka/MongoDB sin omisiones. Se transportaron tanto 621 promociones
oficiales como cinco vistas municipales verificadas. Las ejecuciones no son
equivalentes a un scraping vigente de alquileres. Ver
[evidencias locales](evidencias_2026-11-11/) y [contraste docente](CONTRASTE_DOCENTE.md).

El informe corregido se genera con `python main.py --pdf`; lee métricas,
diccionario, figuras y documentos verificados, sin deserializar el modelo.
Produce Markdown, PDF paginado y manifiesto de fuentes en `data/informe_entrega/`.
Los generadores históricos ahora delegan en él. El PDF maestro adjunto original
es contexto histórico y no debe confundirse con el informe vigente.

CI remoto del incremento **1c7c45e**: [BDA
43008668473](https://github.com/markelalvarez0/Reto-1/actions/runs/43008668473)
y [n8n 43008668600](https://github.com/markelalvarez0/Reto-1/actions/runs/43008668600),
ambos **Success**. Los cuatro PDF versionados contienen ahora el informe corregido
de 24 páginas. No se confunden con el PDF histórico adjunto de la conversación.

## Objetivo confirmado

Estimar alquileres con datos observados y orientar al joven mediante reglas. El
modelo conservado estima **renta media municipal trimestral**, no el precio de un
piso ni la solvencia individual. Su selección y evaluación final están cerradas;
abrir la aplicación o ejecutar la auditoría no vuelve a entrenar ni evalúa test.

## Incremento de desarrollo: acceso, integridad y comparación territorial

- `main.py` abre Streamlit y el modelo conservado; conserva los códigos de error,
  ejecuta la suite completa y resuelve rutas desde la raíz del proyecto.
- `python main.py --audit --json` verifica los seis originales, los 88 artefactos
  conservados, el informe de test único y las tres copias académicas de `src/`.
  No carga archivos pickle/joblib ni ejecuta predicciones.
- `scripts/analyze_municipal.py` carga estadísticas EMAL en `rentas_emal` y el
  contexto OVJ en `contextos_ovj`, conservando las huellas. La identidad incluye
  captura, INE y trimestre. La recarga no duplica documentos ni borra colecciones.
- Tres agregaciones MongoDB producen comparación municipal, serie temporal y
  resumen territorial. El `$lookup` cruza cada observación con el contexto
  autonómico documentado; nunca lo etiqueta como salario municipal.
- `app/territorio.py` lee salidas con manifiesto SHA-256, muestra tablas, gráficos,
  CSV descargable y coordenadas de promociones Visesa. La cartografía base usa
  recursos externos; su disponibilidad no está garantizada en una red restringida.

## Cómo ejecutarlo

Python 3.12. Desde la raíz:

```bash
uv sync --locked --extra dev
uv run --no-sync python main.py --audit --json
uv run --no-sync python main.py --subject bda_sbd --prepare-only

# MongoDB local accesible; MONGO_URI permite cambiar el servidor/puerto.
uv run --no-sync python scripts/analyze_municipal.py
uv run --no-sync python main.py --subject bda_sbd --visual
uv run --no-sync python main.py --subject saa_pia_mia

uv run --no-sync python main.py --test -q
```
```

Resultados municipales y límites (continuación)

```
# Con MongoDB real configurado en MONGO_INTEGRATION_URI:
uv run --no-sync python -m pytest integration_tests/test_mongodb_live.py
integration_tests/test_municipal_live.py -q
```

Las salidas regenerables se guardan en `data/analisis_municipal/`, fuera de Git.
`--simulate-db` permite desarrollo sin servidor y marca el motor como mongomock;
no es evidencia de ejecución en MongoDB real. Se mantiene la alternativa pip
documentada en el README. Las versiones del lock actual no sustituyen el registro
histórico del entorno con el que se entrenó el modelo publicado.

Resultado descriptivo reproducido: 2024 T4

18 municipios con renta publicada en ese trimestre. La serie completa conservada
tiene 19 municipios; no se rellena el dato ausente para completar la tabla.

| Municipio | Renta media mensual | Contratos | Variación interanual | Esfuerzo del
escenario |
|---|---:|---:|---:|---:|
| Vitoria-Gasteiz | 798,57 € | 549 | 2,88 % | 51,06 % |
| Bilbao | 930,31 € | 767 | 6,86 % | 59,48 % |
| Donostia-San Sebastián | 1.147,53 € | 516 | 8,53 % | 73,37 % |

El escenario utiliza 1.564 €/mes, salario neto medio de juventud asalariada de
18–34 años en Euskadi en 2024, incluidas pagas extra (OVJ). Entre estas tres
capitales, la renta media de Vitoria es menor, pero las tres superarían el 30 %
de ese ingreso si una persona pagara íntegramente la renta media municipal.
Esto ilustra una dificultad económica; no demuestra qué proporción de jóvenes
puede emanciparse ni representa todos los hogares o contratos actuales.

Los indicadores son renta mensual, equivalente anual, número de contratos,
variación interanual cuando existe el dato previo, esfuerzo del escenario y
diferencia frente al umbral analítico del 30 %. Este umbral no concede ni deniega
subvenciones. Las medias territoriales son simples entre municipios cubiertos,
no medias de todos los contratos de cada provincia.

Rúbricas: evidencia y trabajo que permanece

Asignatura	Evidencia utilizable	Pendiente para cerrar toda la entrega
BDA	Capturas oficiales y vistas municipales verificadas en HDFS/Kafka/MongoDB reales,
con recarga idempotente y CI correcto en 1c7c45e.	Muestra vigente de alquiler libre
obtenida por scraping. El HTML OVJ no equivale a anuncios.
SBD	Modelado, limpieza, consultas, agregaciones e índices con mongod real; comparación
EMAL/OVJ y transporte de vistas incluidos en evidencias.	Ensayo de la demostración y
defensa. Las consultas de anuncios se prueban con fixtures identificados.
SAA	EDA, cinco familias, MLP y experimentos, SHAP y test único conservado. Contraste
docente documentado.	Defensa individual. sklearn no se presenta como Keras; no se cambia
el ganador mirando test.
PIA	Pipeline, MLflow, notebooks, Streamlit y PDF corregido desde artefactos verificados.
Capturas y prueba de reapertura actuales.	Validación de la versión nueva en Windows si se
entrega para esa plataforma.
MIA	Ocho ramas n8n y recorrido Streamlit → consentimiento → ficha local, con
persistencia e idempotencia reales.	Ensayo humano final. Ninguna prueba tramita ayudas ni
comunica con personas.

Los 33 tests históricos fueron ampliados en `dev-jon`; contar tests no sustituye
revisar qué prueban. Los avisos de convergencia de mini-tests no son resultados de
entrenamiento de entrega. Los originales, los artefactos ML y el test final
permanecen conservados byte a byte.

Se ha preparado [defensa de 15 minutos y preguntas](DEFENSA.md). Permanecen

Resultados municipales y límites (continuación)

pendientes el scraping autorizado de anuncios individuales y el ensayo humano. La integración en test/main espera validación
integral, como autorizó el usuario. Las cifras históricas de la tabla anterior deben leerse junto a esta actualización, no como trabajo aún sin implementar.

Contratos BDA y evidencia

````
# BDA: ingesta trazable y transporte de las capturas oficiales

## Continuación validada el 11 de noviembre de 2026

En esta máquina se ejecutaron cinco pruebas end-to-end sin skips con Hadoop
3.4.2 lean (SHA512 oficial), Java 11, Kafka 3.9.1 y MongoDB 7.0.
La preparación volvió a aceptar 621 registros (215 Visesa y 406 Etxebide),
rechazando dos de los 623 originales por municipio ambiguo. Llegaron y se
persistieron los 621, con procedencia y recarga idempotente.
Ver [XML](evidencias_2026-11-11/integracion.xml) y
[resumen local](evidencias_2026-11-11/bda-resumen.json).

El análisis territorial se exportó con MongoDB real y se transportaron sus cinco
vistas por separado. `scripts/transport_municipal.py` verifica el manifiesto,
almacena procesados en HDFS, recupera y compara bytes, publica lo recuperado en
Kafka y persiste en `vistas_municipales` antes de confirmar offsets.
La identidad `(archivo, sha256)` permite reentrega sin duplicar. Los mensajes son
vistas agregadas, no HousingAd ni viviendas nuevas. Una nueva huella conserva
otra versión. La prueba repite todo el transporte y comprueba cinco documentos.

```bash
python scripts/analyze_municipal.py --output-dir evidencia_bda_integracion/municipal
python scripts/transport_municipal.py --input-dir evidencia_bda_integracion/municipal
--report evidencia_bda_integracion/transporte_municipal.json
```

MONGO_URI debe seleccionar el mismo servidor para ambos comandos. El workflow
registra MONGO_INTEGRATION_URI para la prueba municipal, exporta las vistas,
ejecuta la auditoría y rechaza XML vacíos, errores, fallos, skips y ausencia de
las pruebas municipales. Archiva los resultados y sus huellas. Esta validación
local precede al commit; no se presenta como un CI remoto de checkout limpio.

## Alcance de esta correccion

Este recorrido usa los originales conservados en `data/capturas_oficiales/`, no anuncios
inventados. Integra los catalogos de promociones de Visesa y Etxebide, el nomenclator INE y
el contexto salarial publicado por el OVJ. **Una promocion no equivale a una vivienda
disponible ni a un anuncio individual de alquiler.**

Se distinguen codigo implementado, pruebas con fixtures y pruebas con capturas reales. La
evidencia de este recorrido, cuando la validacion termina correctamente, queda en
`docs/evidencias_bda/resumen.json` y en el workflow `bda-candidate.yml`.

## Contratos de los datos

| Fuente | Contrato y tratamiento |
|---|---|
| Visesa | La captura verificada usa `rows`. El lector conserva esa compatibilidad y admite
por separado `promociones`, sin convertir precios de locales o garajes en rentas. Un
contrato desconocido provoca un error, no un exito con cero filas. |
| Etxebide | CSV con `;`, codificacion ISO-8859-1 y columnas `ID_PROM`, `Cod_Prov`,
`CODMUNI`, `Municipio` y `Reg Acceso`. No publica una renta mensual individual en esas
columnas. |
| INE | XLSX con hojas provinciales: se leen `01`, `20` y `48`. Se concatenan CPRO (2
digitos) y CMUN (3), sin incluir DC. |
| OVJ | Se extrae el salario del parrafo correspondiente a 18-34 anos, 2024, neto mensual e
incluidas pagas extraordinarias. Se conserva como referencia de Euskadi, no como salario de
cada municipio. |

Los nombres historicos de Etxebide se resuelven por el codigo publicado, si existe en el
catalogo INE verificado y no contradice un nombre reconocido. Se conservan el nombre
original y el metodo de correspondencia. No se aceptan coincidencias parciales ni se asigna
````

Contratos BDA y evidencia (continuación)

Bilbao como localidad por defecto.

Integridad de los originales

Se detecto que `* text=auto` normalizaba CRLF a LF al incorporar el CSV de Etxebide y el HTML del OVJ a Git. El contenido textual seguia siendo equivalente, pero la huella de los bytes ya no coincidia con el manifiesto.

La correccion recupera los bytes exactos del artefacto original de captura, sin recapturar la fuente ni modificar las huellas esperadas. La regla `/data/capturas_oficiales/** -text -diff` impide nuevas conversiones. Antes de procesar se verifica SHA-256 de cada original; una diferencia detiene la carga.

Recorrido reproducible

1. Verificar manifiesto, originales e INE; preparar `vivienda_oficial.json` y `municipios_oficiales.json` fuera de la carpeta de originales.
2. Conservar en HDFS tanto las capturas originales como los derivados en rutas `raw` y `processed` separadas. Recuperar cada fichero y comparar bytes y SHA-256.
3. Publicar registros `HousingAd` con fuente, tipo de registro, procedencia y lote de ingesta. Las claves de particionado son codigos municipales.
4. Consumir solo el lote esperado y confirmar el desplazamiento de Kafka despues de la persistencia en MongoDB.
5. Guardar por `(fuente, id_origen)` sin borrar colecciones ni reiniciar contadores. Comprobar que la procedencia sobrevive al transporte.
6. Exportar recuentos de registros de promociones por municipio y fuente. Las estadisticas de alquiler individual excluyen `tipo_registro=promocion`.

Los recuentos de dos catalogos **no deben sumarse como viviendas unicas**: puede haber promociones presentes en ambas fuentes. Los importes y superficies desconocidos siguen siendo nulos. El salario autonomico se guarda en `contexto_autonomico.salario_ovj`; el campo de salario municipal no se rellena con ese valor.

Ejecucion

Desde la raiz del repositorio, con el entorno de `requirements-ci.txt` instalado:

```bash
# Solo comprobar y preparar originales; no requiere servicios.
python "BDA + SBD/main.py" --prepare-only

# Recorrido real: requiere HDFS, Kafka y MongoDB accesibles.
python "BDA + SBD/main.py" --pipeline

# Configuracion explicita del lote y salidas.
python scripts/run_pipeline.py --snapshot-dir data/capturas_oficiales --raw-dir
data/bda/prepared --output-dir data/bda/processed --db-name reto1_vivienda --interval-
seconds 0.1

# Demostracion separada, con datos y servicios simulados identificados.
python scripts/run_pipeline.py --demo

# Regresion y sincronizacion de codigo.
python -m pytest tests/ -q
python scripts/sync_sources.py --check
```

El comando de la raiz `python main.py --subject bda_sbd --pipeline` delega en el mismo recorrido. No es necesario copiar otro `src/` al PYTHONPATH. Las conexiones usan `MONGO_URI`, `MONGO_DB`, `HDFS_NAMENODE_HOST`, `HDFS_NAMENODE_PORT`, `HDFS_USER`, `KAFKA_BOOTSTRAP_SERVERS` y las opciones del cliente; consultar `.env.example`.

Contratos BDA y evidencia (continuación)

Verificacion y limites pendientes

La validacion usa Hadoop, Apache Kafka y MongoDB temporales en GitHub Actions. Comprueba los originales, el transporte, la persistencia, los recuentos, las uniones municipales y una recarga idempotente. No modifica el modelo ML ni vuelve a evaluar su test reservado.

Esto **no acredita todavia** una muestra vigente de alquiler libre obtenida por scraping ni la descarga verificada de todos los indicadores Udalmap. Los scrapers y clientes antiguos que no intervienen en este recorrido requieren su propia revision, incluido el cliente OVJ heredado con constantes. No deben usarse sus salidas historicas como evidencia observada. La comparación actual emplea EMAL y OVJ verificados, no esas constantes.

El mapa histórico no se abre como si contuviera estos resultados. El visor territorial actual está comprobado con resultados de MongoDB real y muestra promociones localizadas, no ofertas disponibles. Permanecen pendientes los indicadores de mercado libre basados en anuncios individuales.

Reglas y automatización local

Prioridad descendente; primera regla aplicable. Seguridad: datos inválidos o fuera de cobertura → revisión. Después se revisan vigencia normativa, esfuerzo crítico, requisitos, documentación, alternativas, gastos de entrada y consulta informativa. Fallback: información general.

Streamlit exige consentimiento y envía solo UUID, regla, decisión y referencia estimada. n8n separa ocho ramas de acción; el receptor local persiste una ficha por UUID. Repetir el mismo evento no duplica; reutilizar UUID con contenido diferente se rechaza. Solo se considera éxito con confirmación de persistencia. No se envían mensajes reales.

Las reglas normativas deben contrastarse con la convocatoria aplicable; la referencia no es una resolución administrativa ni la renta media una oferta disponible.

Reproducción y defensa

```
# Defensa de 15 minutos

Ensayar con los artefactos conservados; no entrenar ni volver a evaluar test.
No presentar una ejecución histórica como si acabara de realizarse.

| Minutos | Contenido y demostración |
|---|---|
| 0–2 | Problema, usuario y límite. Bilbao 2024T4: renta 930,31 EUR frente a salario
regional OVJ de 1.564 EUR; escenario 59,48 %, no salario individual ni municipal. |
| 2–4 | Fuentes originales y manifiesto, INE y rechazos. Promociones no son ofertas
disponibles. Mostrar HDFS raw/processed y recuperación idéntica. |
| 4–6 | Kafka → MongoDB: lote, procedencia, persistencia antes del offset, recarga sin
duplicados. Distinguir las vistas EMAL de los mensajes HousingAd. |
| 6–8 | Comparar las tres capitales y mostrar tabla, serie y mapa. Seis indicadores; media
territorial simple y cobertura limitada. |
| 8–10 | Datos y pipeline ML, división temporal 601/75/92, comparativa MLflow y MLP_2_capas
frente a persistencia. Explicar la pequeña diferencia. |
| 10–12 | SHAP de validación y test sellado (MAE 28,67 EUR); abrir la app sin recalcular.
Predecir un caso declarado como demostración. |
| 12–14 | Reglas por prioridad, consentimiento, n8n y ficha local. Repetir UUID para mostrar
idempotencia sin enviar comunicaciones reales. |
| 14–15 | Limitaciones, requisitos pendientes y conclusión vinculada a los indicadores. No
afirmar cumplimiento integral sin scraping verificado. |

## Reproducción

Desde la raíz con Python 3.12: `uv sync --locked --extra dev`.
`python main.py --audit --json` comprueba integridad;
`python main.py --test -q` ejecuta la suite.
`python scripts/analyze_municipal.py` requiere MongoDB real en MONGO_URI;
`--simulate-db` solo es desarrollo identificado.
`python main.py --subject bda_sbd --visual` abre el visor;
`python main.py --subject saa_pia_mia` abre el modelo conservado.
`python main.py --pdf` genera un informe desde artefactos sin reentrenar.

Los servicios HDFS, Kafka, MongoDB y n8n deben arrancarse expresamente.
Ver `.github/workflows/bda-candidate.yml`, `infra/start_hdfs_ci.sh` y
`n8n/compose.yml`. Los XML con skips no acreditan integración real.

## Preguntas individuales para preparar

- ¿Por qué regresión? El objetivo observado es una renta continua agregada; no hay etiquetas
personales de solvencia.
- ¿Dónde se ajustan imputadores y escaladores? En train y dentro de cada fold temporal,
mediante Pipeline.
- ¿Por qué persistencia? Lag1 y objetivo tienen fuerte correlación; un modelo complejo debe
compararse con esa referencia.
- ¿Por qué no usar SMOTE sobre euros? Genera muestras para clases; no corresponde al
objetivo continuo. Se conserva un experimento auxiliar separado.
- ¿Por qué no cambiar a Keras tras ver test? La selección está cerrada. Los ejercicios
muestran otra biblioteca, pero el enunciado no la fija.
- ¿Qué explica SHAP? Contribuciones del modelo en validación, no causalidad ni derechos
administrativos.
- ¿Cuándo se confirma Kafka? Después de persistencia y verificación. Una caída previa
permite reentrega e inserción idempotente.
- ¿Qué protege un índice único? La identidad de cada dato; no convierte catálogos de dos
fuentes en viviendas únicas.
- ¿Qué significa el esfuerzo? Escenario renta/ingreso regional, no proporción de jóvenes
capaces de emanciparse.
- ¿Qué hace n8n? Guarda acciones locales distintas con consentimiento e idempotencia; no
concede ayudas.
- ¿Qué queda sin acreditar? Scraping vigente de anuncios individuales y cualquier
```

Reproducción y defensa (continuación)

integración cuyo XML/evidencia actual falte.

Apéndice: agregaciones municipales completas

Código canónico completo; no se recorta para ajustar a una página.

```
"""Análisis descriptivo EMAL + OVJ en MongoDB, separado del entrenamiento.

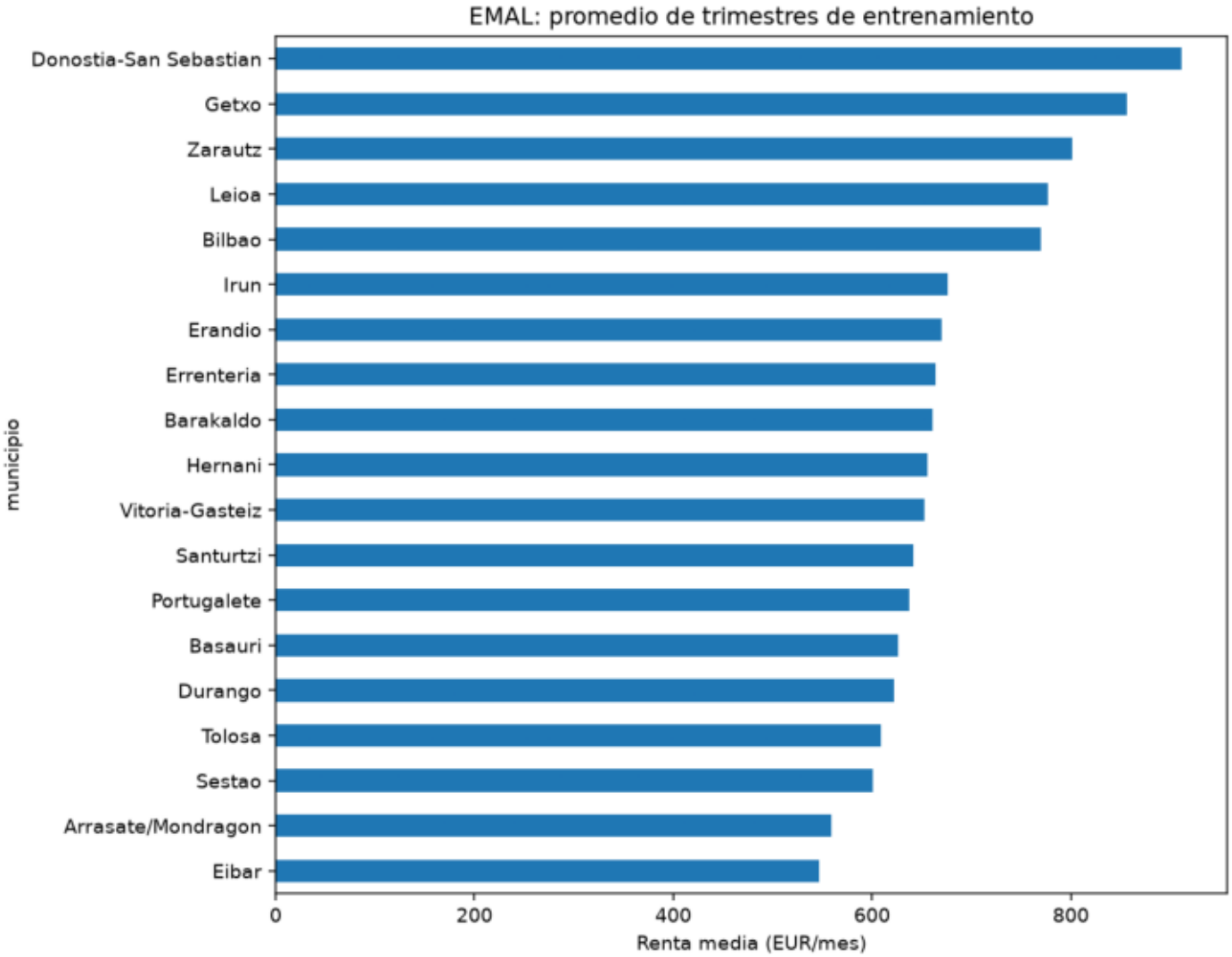
Unidad: municipio/trimestre, nunca anuncio ni hogar. El esfuerzo es un escenario
con salario autonómico de 2024, no una estimación del salario municipal.
"""
import json
import math
from pathlib import Path
import pandas as pd
from src.ml_data import read_emal
from src.extraction.source_snapshots import verify_snapshot, MunicipalityCatalog,
parse_ovj_salary


def analyze(db, snapshot_dir, year=2024, quarter=4):
    """Carga idempotente por huella y exporta tres agregaciones del mismo periodo."""
    root = Path(snapshot_dir)
    sources = verify_snapshot(root)
    frame, metadata = read_emal(root)
    salary_source = sources["ovj_salario_2024.html"]
    salary = parse_ovj_salary((root / salary_source["archivo"]).read_bytes(), salary_source)
    if year != int(salary["periodo"]) or quarter not in (1, 2, 3, 4):
        raise ValueError("Comparar con el salario OVJ del mismo año: 2024, trimestre 1 a 4")
    snapshot = sources["emal_municipios.xlsx"]["sha256"]
    context_id = salary_source["sha256"]
    period = year * 4 + quarter - 1
    if frame.loc[frame.periodo.eq(period)].empty:
        raise ValueError("El trimestre elegido no tiene observaciones")
    db.contextos_ovj.update_one({"_id": context_id}, {"$set": salary}, upsert=True)
    db.rentas_emal.create_index([("captura", 1), ("codigo_municipio", 1), ("periodo", 1)],
unique=True)
    selected = frame[["codigo_municipio", "municipio", "provincia", "anio", "trimestre",
"periodo",
                      "renta_media_eur", "numero_contratos", "renta_lag4"]]
    for row in selected.astype(object).where(selected.notna(), None).to_dict("records"):
        identity = {"captura": snapshot, "codigo_municipio": row["codigo_municipio"],
"periodo": row["periodo"]}
        db.rentas_emal.update_one(identity, {"$set": {**row, "contexto_id": context_id,
"fuente": metadata["fuente"]}}, upsert=True)
    match = {"captura": snapshot, "periodo": period}
    comparison = list(db.rentas_emal.aggregate([
        {"$match": match},
        {"$lookup": {"from": "contextos_ovj", "localField": "contexto_id", "foreignField":
"_id", "as": "contexto"}},
        {"$unwind": "$contexto"},
        {"$project": {"_id": 0, "codigo_municipio": 1, "municipio": 1, "provincia": 1,
"anio": 1, "trimestre": 1,
            "renta_media_eur": 1, "numero_contratos": 1,
            "renta_anual_equivalente_eur": {"$multiply": ["$renta_media_eur", 12]},
            "variacion_interanual_pct": {"$cond": [{"$gt": ["$renta_lag4", 0]},
                {"$multiply": [{"$subtract": [{"$divide": ["$renta_media_eur",
"$renta_lag4"]}, 1]}, 100]}, None]},
            "salario_referencia_autonomico_eur": "$contexto.valor",
            "esfuerzo_escenario_pct": {"$multiply": [{"$divide": ["$renta_media_eur",
"$contexto.valor"]}, 100]},
            "diferencia_umbral_30_eur": {"$subtract": ["$renta_media_eur", {"$multiply":
["$contexto.valor", 0.3]}]},
        }},
        {"$sort": {"renta_media_eur": 1, "codigo_municipio": 1}},
    ]))
```

Apéndice: agregaciones municipales completas (continuación)

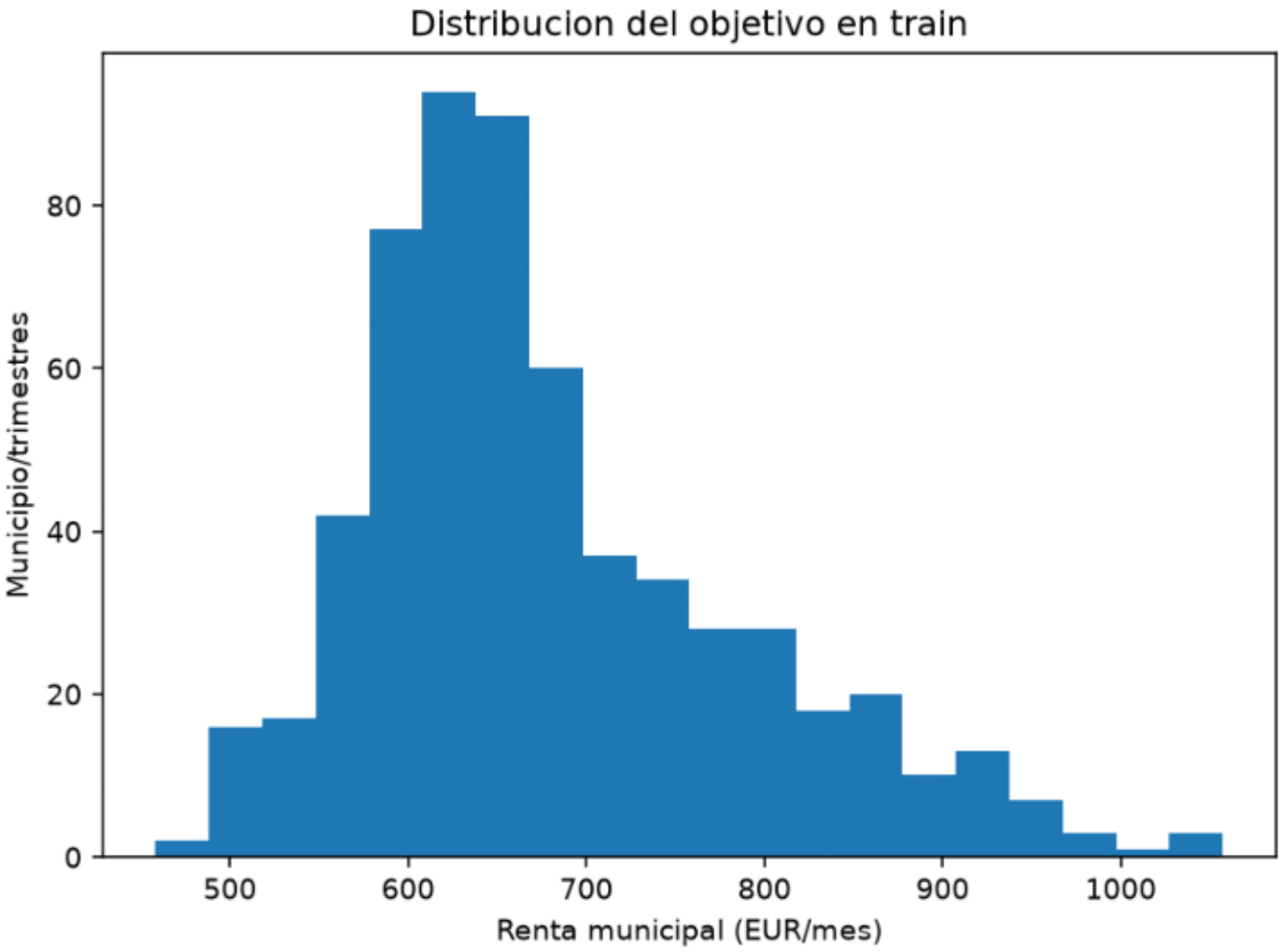
```
    # Media simple entre municipios: no confundirla con renta de todos los contratos.
    territory = list(db.rentas_emal.aggregate([
        {"$match": match}, {"$group": {"_id": "$provincia", "municipios": {"$sum": 1},
            "media_simple_rentas_municipales_eur": {"$avg": "$renta_media_eur"},
            "contratos_publicados": {"$sum": "$numero_contratos"}}},
        {"$sort": {"_id": 1}},
    ]))
    series = list(db.rentas_emal.aggregate([
        {"$match": {"captura": snapshot, "anio": {"$lte": year}}},
        {"$project": {"_id": 0, "codigo_municipio": 1, "municipio": 1, "anio": 1,
"trimestre": 1,
                      "periodo": 1, "renta_media_eur": 1, "numero_contratos": 1}},
        {"$sort": {"periodo": 1, "codigo_municipio": 1}},
    ]))
    catalog = MunicipalityCatalog.from_snapshot(root, sources)
    points = []
    for row in json.loads((root / "visesa.json").read_text(encoding="utf-8"))["rows"]:
        code = catalog.resolve(row.get("poblacion", ""))
        try:
            lat, lon = float(row["latitud"]), float(row["longitud"])
        except (TypeError, ValueError, KeyError):
            continue
        if code and math.isfinite(lat) and math.isfinite(lon) and 42 <= lat <= 44 and -4 <=
lon <= -1:
            points.append({"id_promocion": str(row.get("idPromocion")), "codigo_municipio":
code,
                           "municipio": catalog.records[code]["municipio"], "lat": lat,
"lon": lon})
    if len(comparison) < 3:
        raise ValueError("La comparación necesita al menos tres municipios")
    return {"comparativa_municipal": comparison, "resumen_territorial": territory,
            "serie_trimestral": series, "mapa_promociones": points,
            "metadatos": {"anio": year, "trimestre": quarter, "unidad":
"municipio/trimestre",
                "fuentes": {name: sources[name] for name in ("emal_municipios.xlsx",
"ovj_salario_2024.html", "visesa.json", "ine_2026.xlsx")},
                "salario": salary,
                "limites": ["El esfuerzo es un escenario con salario autonómico, no una
medida de cada hogar.",
                    "La renta anual equivalente multiplica por 12 la renta mensual: no suma
contratos anuales.",
                    "El umbral del 30% es analítico, no un requisito legal de subvención.",
                    "El mapa localiza promociones Visesa; no representa ofertas disponibles
ni centros municipales.",
                    "La media territorial es simple entre los municipios cubiertos; no
representa todos los contratos.",
                    metadata["advertencia"]]}}
```

01_barras.png



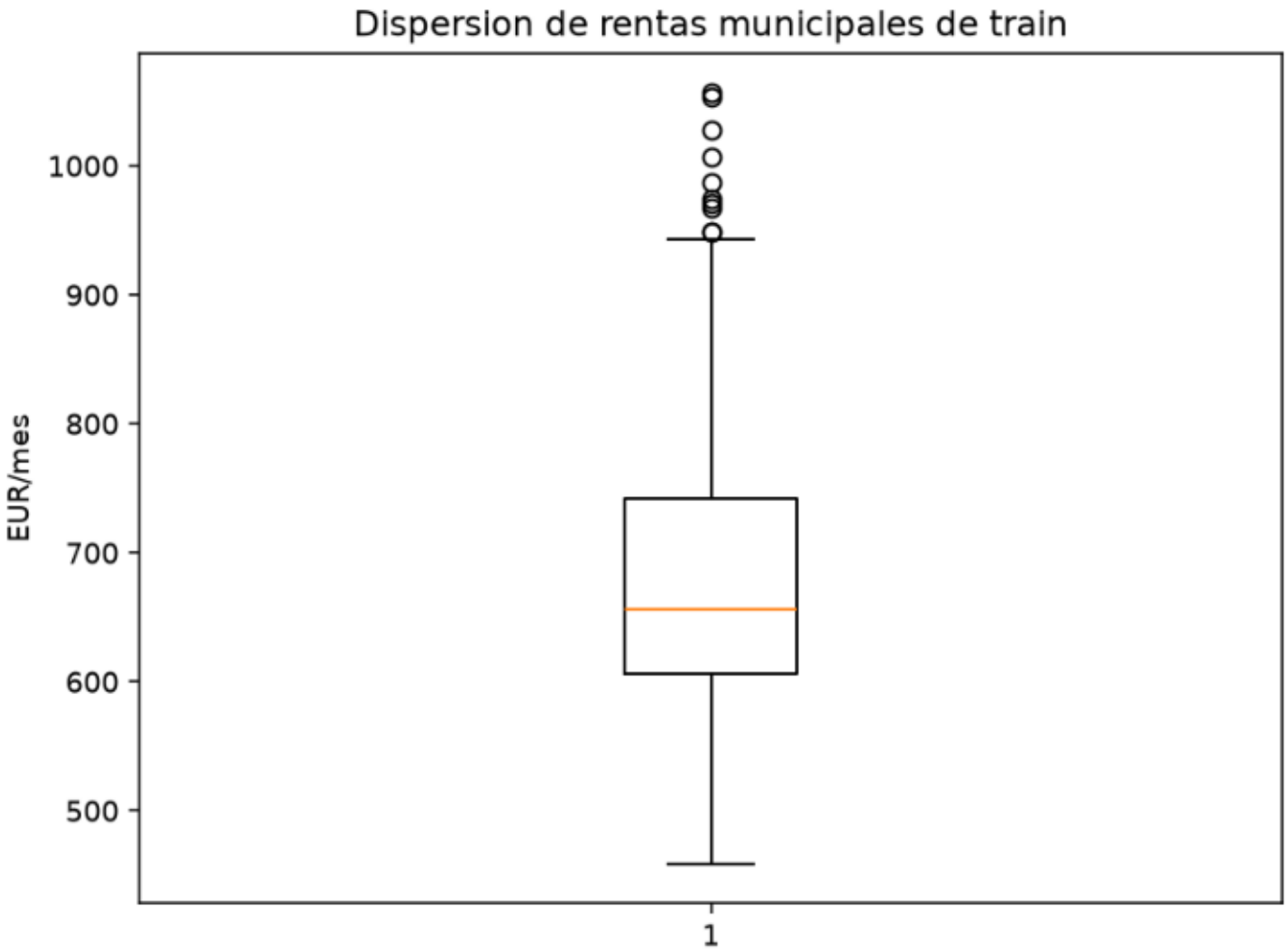
Donostia-San Sebastian presenta la mayor media de train (911.58 EUR). Es una media de agregados trimestrales, no ponderada por contratos.

02_histograma.png



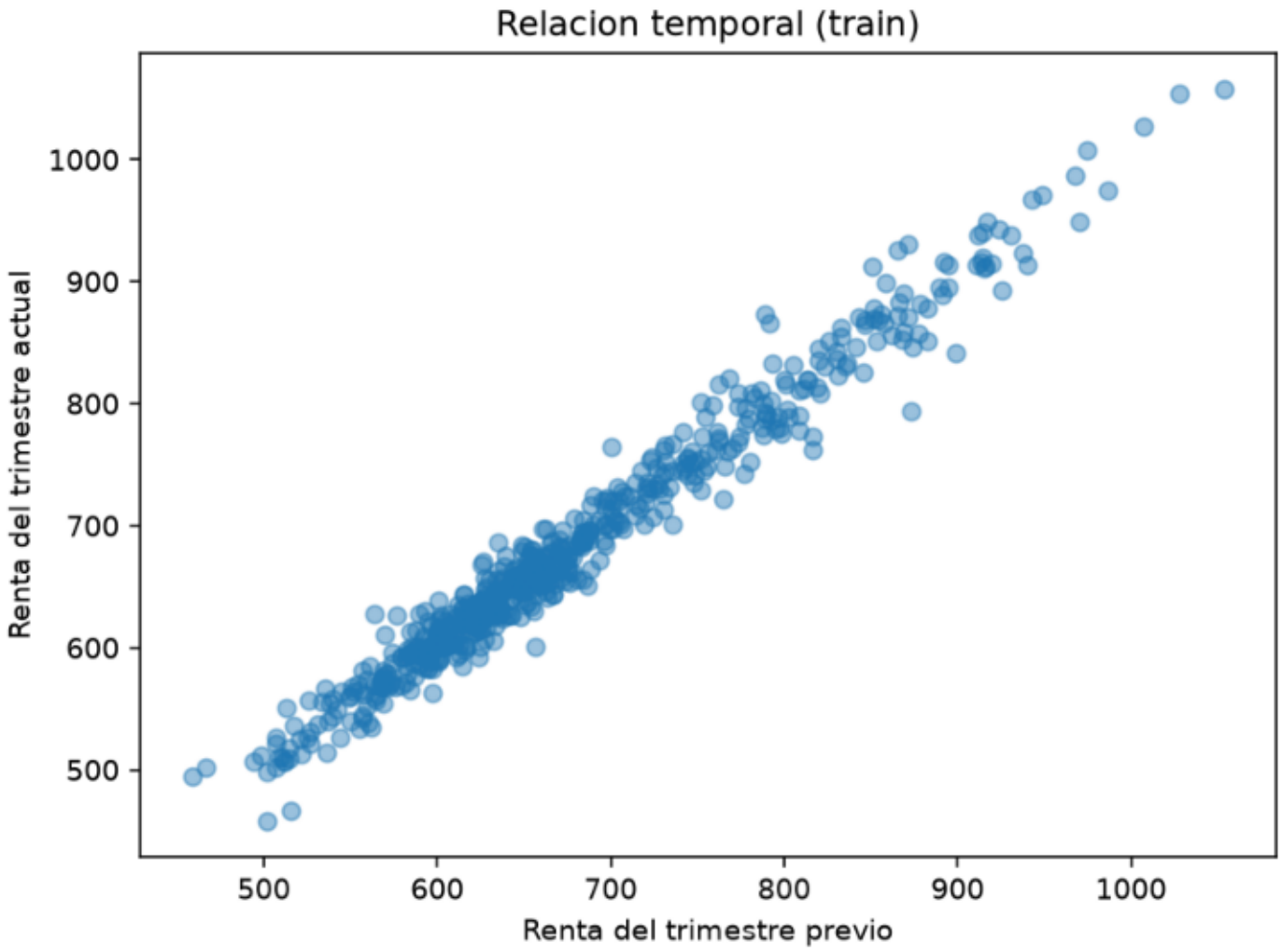
La mediana de train es 656.00 EUR. El objetivo es una renta municipal observada, no la viabilidad de una persona.

03_boxplot.png



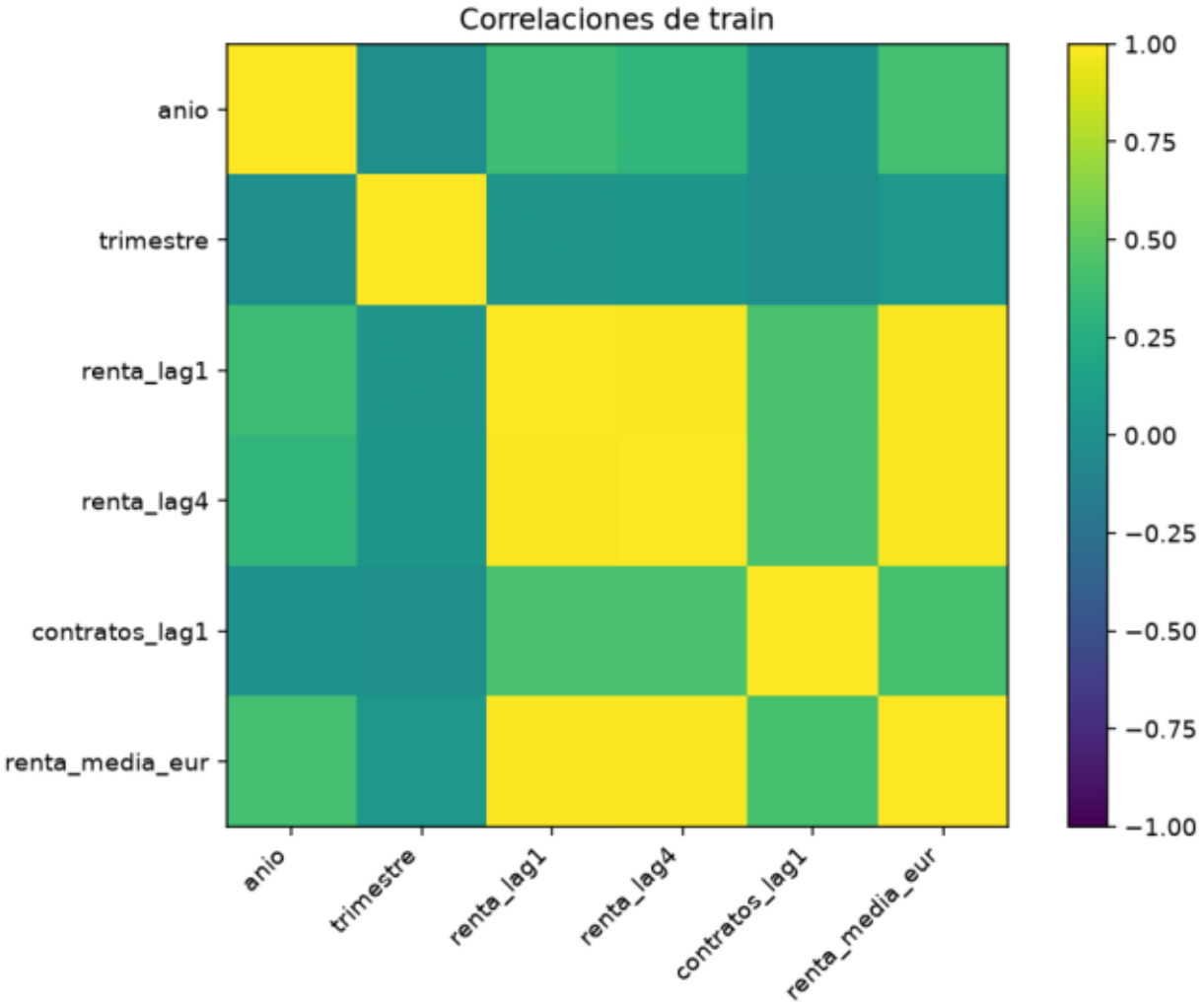
IQR = 136.13 EUR. No se eliminan precios altos solo por ser extremos: pueden reflejar diferencias territoriales reales.

04_dispersion.png



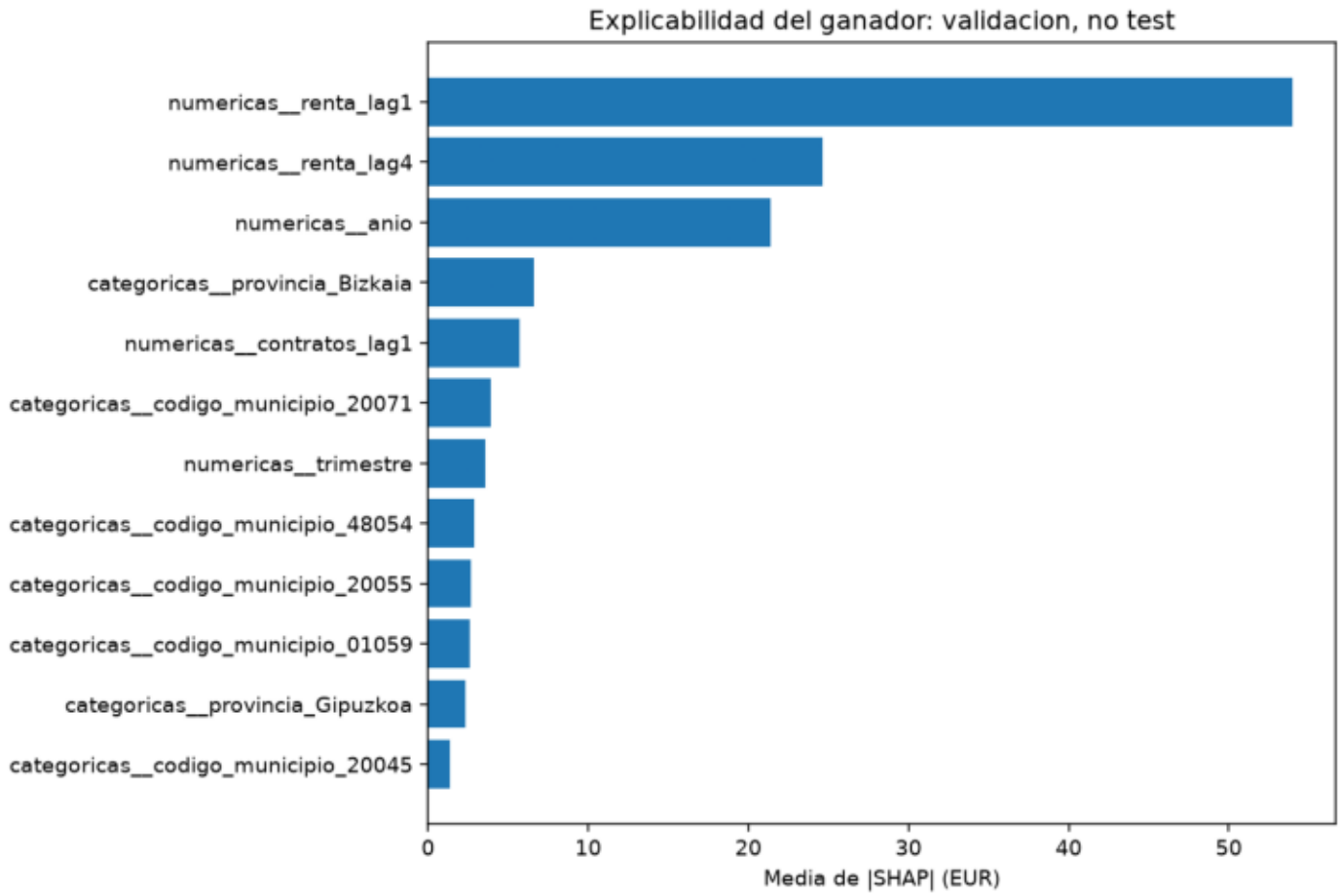
Correlacion lag1/objetivo = 0.985. La persistencia debe ser un baseline; correlacion no demuestra causalidad.

05_correlacion.png



Los retardos recogen persistencia. Ni la renta actual, ni su precio/m2, ni una categoria derivada del objetivo se incluyen entre las entradas.

shap_validacion.png



SHAP de validación; no causal, fondo de train.

test_real_predicho.png

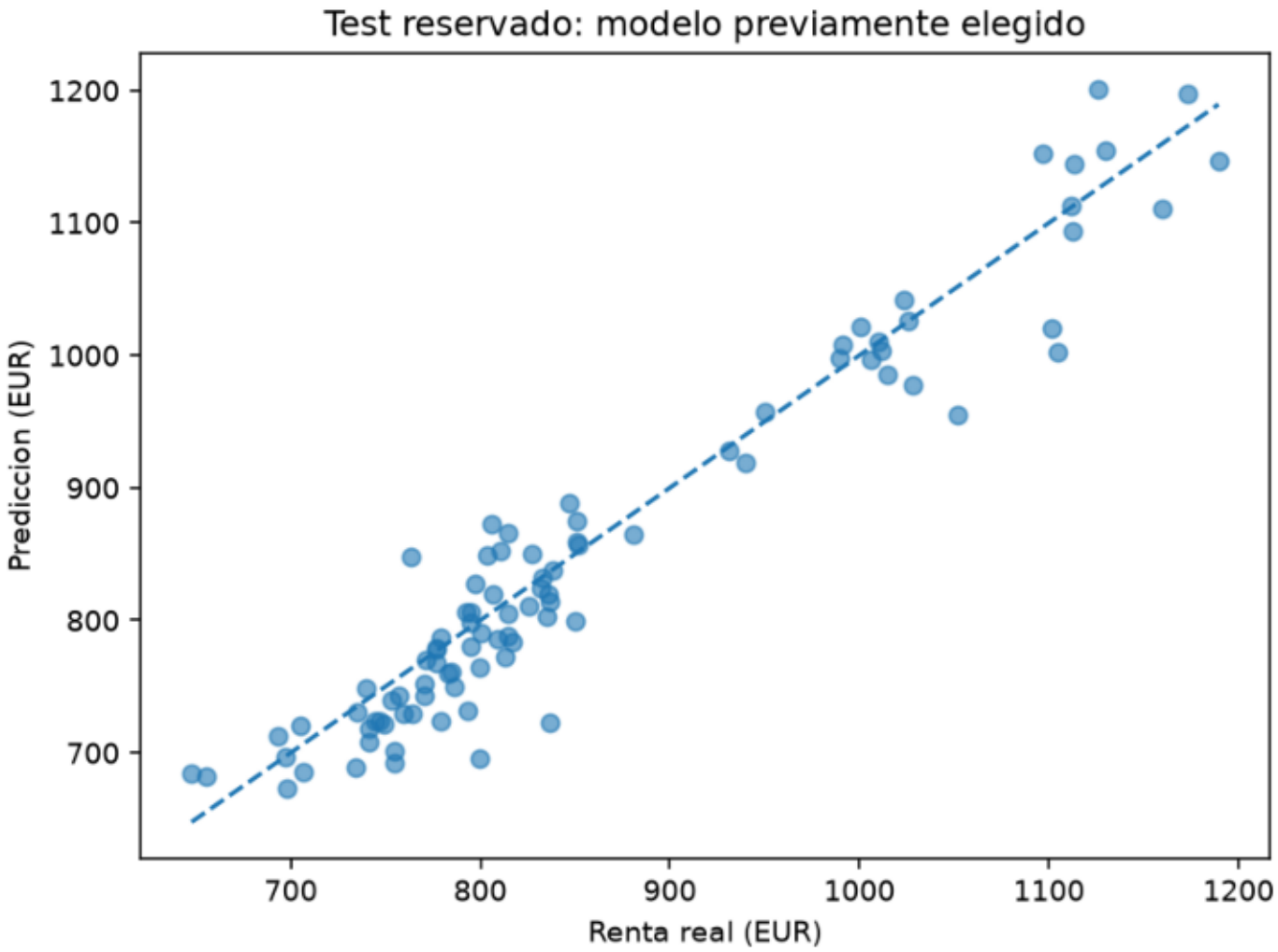


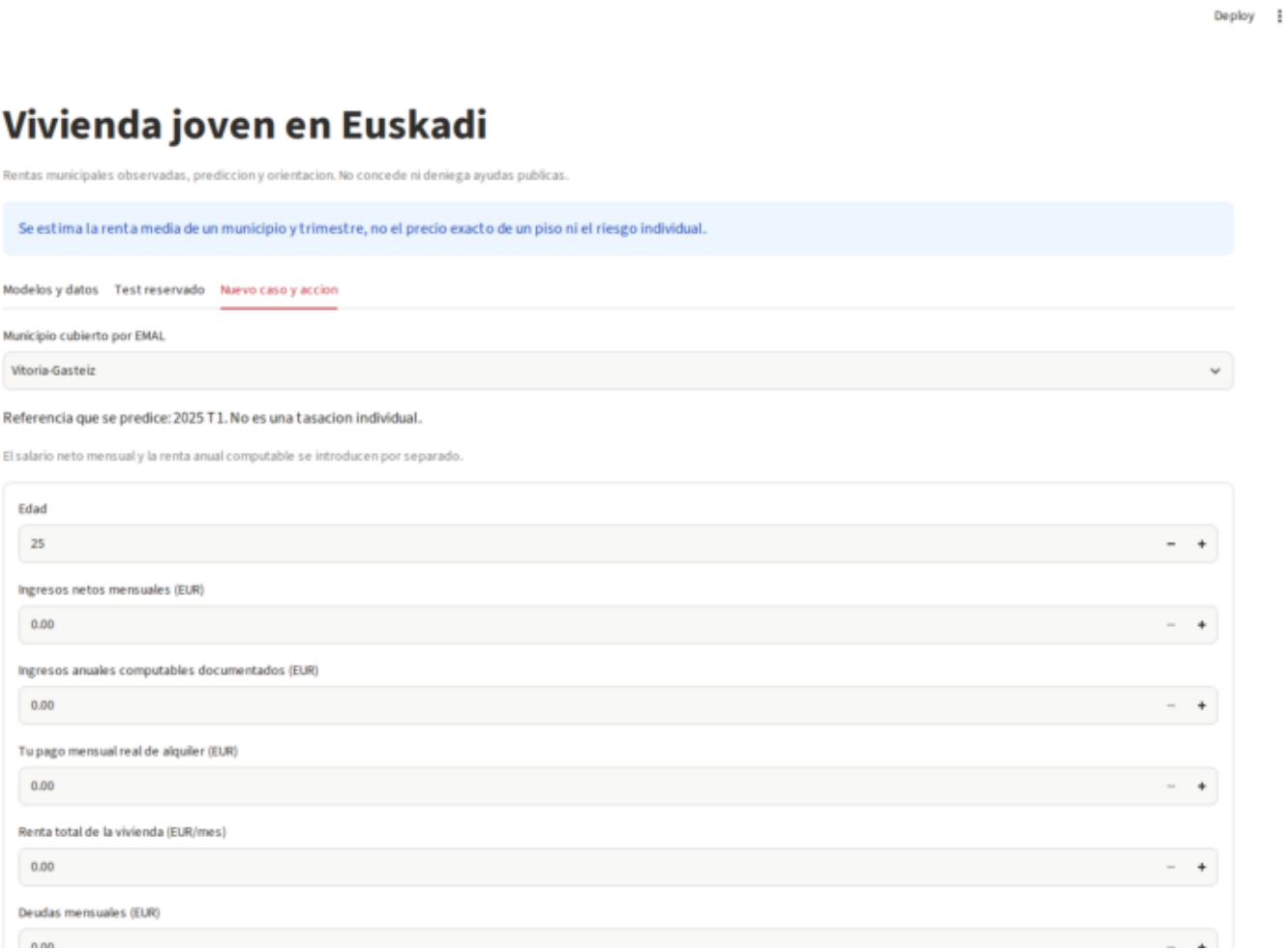
Figura conservada de la única evaluación final; no se recalcula.

mlflow-experimentos.png



Captura histórica conservada del bloque ML; no nueva ejecución.

streamlit-formulario.png



Captura histórica conservada del bloque ML; no nueva ejecución.

streamlit-modelos.png



Captura histórica conservada del bloque ML; no nueva ejecución.